Repository Review Report

Submission Details

- Reviewer: Tarun Raj
- Repo Owner: Nihal Uikey
- Original Repository: https://github.com/Nihaluikey6488/kodex-Ecommerce-backend
- Forked Repository: https://github.com/tarun5004/kodex-Ecommerce-backend
- Review Branch: tarun
- Pull Request Link: To be added after opening the PR from ‘tarun5004:tarun‘ to ‘Nihaluikey6488:main‘
- Documentation File: ‘REVIEW.md‘

Overview

This repository is an Express and MongoDB backend for a small e-commerce product management API. It includes user authentication, JWT-based protected routes, product CRUD APIs, ImageKit image uploads, and MongoDB persistence through Mongoose.

The original code had a working beginner-level structure, but several important boundaries were mixed together. Controllers, services, validation, environment configuration, database access, and upload handling were not cleanly separated. The review focused on keeping the same application idea while making the code safer, easier to maintain, and closer to production-style backend practices.

Issues Found

1. Product service used ‘res.status(...).json(...)‘ even though ‘res‘ is not available inside the service layer.
2. Product ownership was broken because products were stored with email, while queries tried to filter using a non-existing ‘id‘ field.
3. Product read, update, and delete operations did not verify that the product belonged to the logged-in user.
4. Product upload logic imported ImageKit incorrectly after the config changed, which could break create/update flows at runtime.
5. Product service called ‘files.map(...)‘ directly, so requests without files could crash.
6. Product validation was mostly inside the service layer instead of request middleware.
7. File uploads had no file type or file size protection.
8. Auth responses could expose user documents too directly, including sensitive fields in earlier flow.
9. Password selection and comparison were not fully hardened in the model/service boundary.
10. JWT verification used raw environment access and did not return a clean 401 for malformed or expired tokens.
11. Auth middleware only handled cookie tokens, while API documentation also referenced Bearer tokens.
12. Environment variables were not documented with an example file.

13. Logger config referenced ‘env‘ without importing it, causing a runtime failure.
14. Database and ImageKit config files contained large blocks of stale commented code.
15. Global error handling did not normalize upload errors or duplicate key errors.

Fixes Applied

1. Added a repository layer for users and products:
- ‘src/repositories/user.repository.js‘
- ‘src/repositories/product.repository.js‘
2. Refactored product service so it no longer depends on Express ‘req‘ or ‘res‘.
3. Changed product ownership to use the authenticated user’s id instead of email.
4. Scoped product list, read, update, and delete operations to the logged-in user.
5. Added ‘express-validator‘ based product validation:
- create product validation
- update product validation
- product id validation
- product query validation
6. Improved product upload handling with Multer limits and image MIME type filtering.
7. Fixed ImageKit usage by calling the named ‘uploadFile‘ helper from config.
8. Preserved existing product images during update when no new files are uploaded.
9. Added ‘.env.example‘ so required configuration is visible.
10. Centralized environment validation with Zod in ‘src/config/env.js‘.
11. Fixed logger configuration and removed stale commented code from config files.
12. Updated JWT middleware to support both cookie tokens and ‘Authorization: Bearer <token>‘.
13. Added clean 401 handling for invalid or expired tokens.
14. Improved global error handling for Multer errors and duplicate MongoDB key errors.
15. Cleaned the server startup file and added graceful shutdown handling for ‘SIGINT‘ and ‘SIGTERM‘.

Setup Instructions

1. Install dependencies:

```
npm install
```

2. Create a local environment file:

```
cp .env.example .env
```

3. Fill these required values in ‘.env‘:

```
MONGO_URI=
JWT_SECRET=
IMAGEKIT_URL_ENDPOINT=
IMAGEKIT_PUBLIC_KEY=
IMAGEKIT_PRIVATE_KEY=
```

4. Start the development server:

```
npm run dev
```

5. Start the server normally:

```
npm start
```

Summary of Changes

The main improvement was moving the backend toward clear responsibility boundaries. Routes now own request validation, controllers coordinate request and response handling, services own business logic, and repositories own database access. Product authorization is now enforced at the query level, which prevents users from accessing or modifying products they do not own.

The auth flow is also safer now. User responses are sanitized, password selection is restricted by default, JWT verification is centralized through validated env config, and token failures return consistent unauthorized responses.

Folder Structure Review: Before vs After

Before Review

The earlier folder structure had the basic Express project shape, but some responsibilities were mixed together:

```
src/
 app.js
 config/
    db.js
    imageKit.js
 controllers/
    auth.controller.js
    product.controller.js
 middlewares/
    auth.middleware.js
    multer.middleware.js
 models/
    products.model.js
    users.model.js
 routes/
    auth.routes.js
    product.routes.js
 services/
    auth.service.js
    product.service.js
 utils/
     apiError.js
     apiResponse.js
     asyncHandler.js
     token.js
```

Main problems in the old structure:

1. Services were directly handling database queries, so business logic and data-access logic were tightly coupled.
2. Product service tried to use Express response logic (‘res.status‘) even though services should not know about ‘req‘ or ‘res‘.
3. Validation was scattered inside service functions instead of being handled before the request reached controllers/services.

4. Upload middleware only stored files in memory and did not check file type, file size, or file count safely.
5. Auth middleware directly verified JWT without clean handling for expired or malformed tokens.
6. Configuration was incomplete because there was no ‘.env.example‘, and required env values were not clearly documented.
7. Security middleware such as Helmet, CORS, and rate limiting was either missing or not centralized in a dedicated middleware file.
8. Product authorization was weak because the code did not clearly separate "find product" from "find product owned by this user."

After Review

The updated structure separates each responsibility more clearly:

```
src/
 app.js
 config/
    cookie.js
    db.js
    env.js
    imageKit.js
    logger.js
 controllers/
    auth.controller.js
    product.controller.js
 middlewares/
    auth.middleware.js
    multer.middleware.js
    security.middleware.js
    validate.middleware.js
 models/
    products.model.js
    users.model.js
 repositories/
    product.repository.js
    user.repository.js
 routes/
    auth.routes.js
    product.routes.js
 services/
    auth.service.js
    product.service.js
 utils/
    apiError.js
    apiResponse.js
    asyncHandler.js
    token.js
 validations/
     auth.validation.js
     product.validation.js
```

What improved in the new structure:

1. ‘validations/‘ keeps request validation rules separate from business logic.
2. ‘validate.middleware.js‘ converts ‘express-validator‘ errors into consistent API errors.
3. ‘security.middleware.js‘ centralizes Helmet, CORS, and rate limiting.
4. ‘repositories/‘ keeps MongoDB queries away from service files.

5. ‘product.repository.js‘ now performs owner-scoped product queries, updates, and deletes.
6. ‘env.js‘ validates required environment variables with Zod before the server runs.
7. ‘cookie.js‘ centralizes cookie settings for auth responses.
8. ‘logger.js‘ centralizes structured logging.
9. ‘multer.middleware.js‘ now protects upload endpoints with MIME type and size validation.

Middleware, Validation, and Security Improvements

Validation Middleware

Validation was added so invalid data is rejected before it reaches controllers and services. This keeps the service layer focused on business rules instead of basic request checks.

Examples:

- Register requires valid ‘name‘, ‘email‘, and ‘password‘.
- Login requires valid ‘email‘ and ‘password‘.
- Product creation/update requires valid ‘name‘, ‘price‘, and optional ‘category‘.
- Product id routes validate MongoDB ObjectId format before querying the database.

Auth Middleware

The old auth middleware only checked a cookie token and directly called ‘jwt.verify‘. The updated middleware:

- accepts token from cookie or ‘Authorization: Bearer <token>‘;
- uses validated JWT config from ‘env.js‘;
- returns a clean ‘401‘ response for missing, malformed, or expired tokens;
- attaches the decoded user payload to ‘req.user‘.

Upload Middleware

The old upload middleware only used memory storage. The updated upload middleware:

- allows only JPEG, PNG, and WEBP images;
- limits each uploaded file to 5 MB;
- limits product uploads to 5 files;
- passes invalid upload errors into the global error handler.

Security Middleware

Security middleware was centralized to make baseline API protection easier to reason about:

- ‘helmet‘ adds common secure HTTP headers;
- ‘cors‘ uses the configured client URL and supports credentials;
- ‘express-rate-limit‘ reduces brute force and spam requests.

Environment and Config

The review added ‘.env.example‘ and strengthened config handling through ‘env.js‘. This makes

setup clearer and prevents the server from starting with missing values like ‘MONGO_URI‘, ‘JWT_SECRET‘, or ImageKit keys.

List of Issues Resolved

1. Removed ‘res‘ usage from product service.
2. Fixed broken product owner filtering.
3. Added owner checks for product read, update, and delete.
4. Fixed ImageKit upload helper usage.
5. Made image upload handling safe when files are missing.
6. Added product request validation middleware.
7. Added upload MIME type and size limits.
8. Improved JWT validation and error handling.
9. Added Bearer token support.
10. Added ‘.env.example‘ and centralized env validation.
11. Fixed broken logger config.
12. Cleaned stale commented code from startup/config files.
13. Added repository layer for cleaner database access.
14. Improved duplicate key and Multer error responses.

Improvement Report: Before vs After Analysis

```
| Area | Before | After |
| --- | --- | --- |
| Product ownership | Products used email and queries used a missing ‘id‘ field | Products are tied to authenti
cated user id |
| Authorization | Any logged-in user could access a product by id | Product queries include both product id and
 owner id |
| Service layer | Mixed Express response logic with business logic | Services return data or throw ‘ApiError‘ |
| Validation | Product validation was manual and scattered | Validation uses ‘express-validator‘ at route level
 |
| Uploads | No file type or size protection | JPEG, PNG, WEBP only, with file size and count limits |
| ImageKit | Import/export mismatch could break uploads | Product service uses the correct ‘uploadFile‘ helper
|
| Auth | JWT verification was direct and fragile | Tokens are verified through centralized env config with clea
n 401 errors |
| Config | Missing documented env example | Added ‘.env.example‘ and Zod env validation |
| Logging | Logger referenced undefined config | Logger now imports validated env config |
| Maintainability | Database queries lived directly in services | Repository files now isolate data access |
```

Improvements

- Better separation of concerns across routes, controllers, services, repositories, models, and config.
- Safer product authorization by scoping database queries to the authenticated user.
- More reliable validation using ‘express-validator‘.
- Safer file uploads with type and size checks.
- Cleaner JWT handling with consistent unauthorized responses.
- More predictable runtime configuration through ‘.env.example‘ and Zod validation.
- Better server reliability through fail-fast DB startup and graceful shutdown.

Future Enhancements

1. Add automated tests for auth and product CRUD flows.
2. Add pagination and search for product listing.
3. Add role-based access control if admin features are needed.
4. Add logout and refresh-token flows.
5. Store ImageKit file ids to support image cleanup when products are deleted or updated.
6. Add API documentation with accurate examples for cookie and Bearer token authentication.
7. Add request logging with correlation ids for easier debugging.
8. Add CI checks for syntax, tests, and dependency audit.